CodeMentor AI

A hybrid code mentoring engine that audits and asks Socratic questions instead of fixing code.

PYTHON 3.12 | 3,700+ TESTS | MYPY --STRICT | NO DOCKER NEEDED | MIT LICENSE

At a Glance

Feature	Description
Languages & Contexts	C, C++, C#, Python, HTML, CSS + Unity & Unreal Engine contexts
Static Rules	41 deterministic rules (syntax, memory leaks, Big-O loop depth, security)
Pre-Validation	tree-sitter (Tier 0) + gcc / clang / Roslyn (Tier 1 syntax check)
LLMs Supported	Claude Code CLI, Codex, Anthropic API, Gemini, Ollama (local)
Interfaces	Windows Desktop (.exe), React Web UI, CLI, REST API + SSE
Evaluation Benchmarks	166 labeled fixtures (dev 96 / test 70), 0% leakage, 100% injection defense

Why CodeMentor AI?

When students ask an LLM "what is wrong with my code?", it usually outputs the corrected solution directly. The student copies the answer without understanding; genuine learning is lost. CodeMentor AI reverses this paradigm:

- **Questions over Solutions:** LLM output never reaches the student unfiltered. Socratic questions pass through a Leakage Guard; answers exposing direct solutions are regenerated (0% leakage).
- **Deterministic Foundation:** Syntax errors exit early without calling LLM (~1 ms). If an LLM provider fails, reports gracefully degrade to static analysis.
- **Progressive Hinting (1 → 3):** Level 1 (Conceptual hint) → Level 2 (Line citation & context) → Level 3 (Directional cue). The answer is never given.
- **Security & Isolation:** User code is treated as strictly untrusted input; compilers run in isolated subprocesses via stdin with POSIX rlimits or Windows Job Objects.

What the Student Experiences

1. Syntax Error (Early Exit, 0 LLM Tokens, ~1 ms)

```
int total = 0 // missing semicolon
```

Mentor Question: "An expression on line 4 appears not to terminate with ';'. What does the compiler expect to find following this statement?"

2. Memory Leak & Static Rule (C-MEM-001)

```
int *tmp = malloc(n * sizeof(int)); // never freed
```

Socratic Question (Level 1): "What happens to the memory allocated during function execution once the function returns, and who is responsible for releasing it?"

Engineering & Evaluation Benchmarks

Metric	Claude Opus (two_pass)	Static Only (No LLM)
Audit Precision / Recall / F1 (dev 96)	**79 / 97 / 87**	100 / 57 / 73
Early Exit Precision / Recall	100% / 100%	100% / 100%
Big-O Complexity Accuracy	97% / 97%	97% / 97%

Solution Leakage	**0%**	—
Prompt Injection Resistance	**100%**	—